

최종 결과 보고서

(2025학년도 1학기)

과제명	웹 기반 병원 관리 시스템		
과목 /담당교수	웹서버프로그래밍/ 정복래교수님		
제출일자	2025. 06. 11		
연구참여자 (담당분야)	팀장 : 이현영(과제 총괄 및 보고서 작성) 팀원 : 강현욱(외부 API 연동 개발 및 테스트) 팀원 : 김민준(기능별 테스트 및 디버깅) 팀원 : 김승호(주요 기능 구현 총괄)		
팀명	슬기로운 정통생활	팀 번호	7조

목 차

1. 설계과제 제목	1
2. 연구목적	1
3. 설계과제의 필요성	1
4. 설계과제의 목표	1
5. 설계과정	5
6. 제작	13
7. 시험	17
8. 평가	20
9. 추진체계	24
10. 설계 추진일정	25
11. 결론	25

1. 설계과제의 제목 :

웹 기반 병원 관리 시스템

2. 연구목적

- 1) 데이터베이스 연동, 외부 API 연동 등 백엔드 개발 과정을 실전처럼 경험
 - 단순한 정적 웹 페이지를 넘어서 실제 사용자 데이터를 처리하고 외부 시스템과의 연계를 포함한 서버 프로그래밍 전반을 직접 설계 및 구현한다.
- 2) 사용자 유형에 따라 기능을 나누고 실제 서비스처럼 기획하고 구현해보는 경험 축적
 - 관리자, 의료진, 환자 등 사용자 역할에 따른 인터페이스 및 권한 구분, 실제 서비스 시나리오 기반의 기획-구현-테스트 전 과정을 실습함으로써 개발자 관점의 기획 역량도 함께 키운다.
- 3) 개인정보 보호, 역할 기반 접근제어, 자동 로그아웃 등 실제 적용 방식을 학습
 - 보안이 중요한 분야인 의료 정보 처리를 주제로 설정함으로써, 비밀번호 암호화, 세션 만료 처리, 접근제어 등 보안 기법을 실제로 적용해보고 그 중요성을 체험하는 것이 목적이다.

3. 설계과제의 필요성

현대의 의료 현장은 디지털 전환이 빠르게 이루어지고 있으며, 병원 내 다양한 정보와 절차를 하나의 웹 기반 시스템으로 통합하고자 하는 요구가 증가하고 있다. 하지만 이러한 시스템은 사용자 역할 분리, 개인정보 보호, 실시간 데이터 처리 등 다양한 기술적 요구사항을 수반한다.

본 설계과제는 최대한 실제 서비스와 유사한 구조를 가진 복합적인 웹 시스템을 직접 설계 및 구현해봄으로써 실무 감각과 문제 해결 능력을 키울 수 있다는 점에서 큰 의의를 가진다.

또한, 병원이라는 민감한 분야를 대상으로 다룸으로써, 보안과 안정성에 대한 고려가 반드시 요구된다.

4. 설계과제의 목표

병원 시스템을 디지털 기반의 단일 웹 시스템으로 통합하여 사용자 중심, 운영 효율성, 시스템 안정성을 모두 갖춘 병원 웹 시스템의 프로토타입을 완성하는 것이 최종 목표이다.

- 1) 프로그램의 구현 : 병원 웹 시스템의 핵심 기능을 모두 구현
- 2) 인터페이스 구현 : 사용자 친화적인 인터페이스 구현 및 사용자 별 화면 구성
- 3) 모듈화 : 기능별 모듈 구분, 미들웨어 및 API 방식 설계로 코드 유지보수성 확보
- 4) 구현하고자 하는 기능

1. 사용자 역할 시스템 : 회원가입 시 사용자(환자, 의료진, 관리자, 서버 관리자) 역할을 설정하고, 역할 기반 접근 제어를 통해 정보 보안과 사용자 편의성을 확보한다.
2. 환자 정보 관리 : 환자의 개인정보 및 진료 기록, 입·퇴원 이력 등을 통합 관리할 수 있는 인터페이스를 제공한다.
3. 입퇴원 및 병실 관리 : 병실 상태와 환자의 입·퇴원 상태를 연동하여 실시간으로 관리할 수 있는 기능을 구현한다.
4. 진료 예약 및 스케줄 등록 : 환자는 진료 예약을 신청하고, 의료진은 스케줄을 등록 및 조정하며, 예약 요청을 수락 또는 거절할 수 있다.
5. 문자 알림 API : 진료 예약 확정, 입·퇴원 일정, 안내 사항 등을 환자에게 SMS로 전송하여 서비스 전달력을 높인다.
6. 보안 시스템 : 비밀번호 암호화, 자동 로그아웃, 접근 권한 제한 등 웹 보안의 기본 요소들을 적용하여 개인정보 보호와 시스템 안정성을 확보한다.
7. 백업 및 복구 시스템 : 주요 데이터 손실에 대비해 백업 기능을 구현하고, 필요 시 데이터를 복구할 수 있는 관리 기능을 탑재한다.

표 1. 본 프로젝트의 현실적 제한요소 항목

현실적 제한 요소들	내 용 (Content)
경제	- 자체 서버 구축 및 운영에 드는 초기 비용이 크며, 보안 강화와 외부 API 사용 시 지속적인 비용 부담이 발생한다. - 특히 병원 규모가 작을수록 시스템 도입 비용 대비 효율성이 낮아, 비용 부담이 더욱 클 수 있다.
편리	- 고령의 의료진이나 디지털 기기에 익숙하지 않은 사용자는 시스템 학습에 어려움을 겪을 수 있다. - 의료진과 환자 각각의 다양한 사용 환경을 모두 만족시키기 위한 대응이 제한적일 수 있다. - 응급 상황에서 웹 기반 시스템의 작동 지연 또는 장애 발생 시 신속한 대응이 어려울 가능성이 있다.
윤리	- 개인정보와 진료 기록 등 매우 민감한 데이터는 철저한 암호화와 접근 통제를 통해 보호되어야 한다.
사회	- 고령자, 장애인, 외국인 환자 등 디지털 접근성이 낮은 사회적 약자 계층이 서비스에서 소외될 우려가 있다.

5. 설계과정

5.1 설계 기초이론

1) 데이터베이스 설계 및 연동

병원 관리 시스템은 환자 정보, 의료진, 병실 상태 등 다양한 데이터를 효율적으로 저장하고 관리해야 하므로 데이터베이스 설계가 필수적이다. 본 과제에서는 MongoDB를 활용하여 문서 지향 데이터 모델을 적용하였으며, Mongoose를 통해 Node.js와 연동하는 방법을 학습했다.

2) 웹 서버 및 API 개발 기법

Express 프레임워크를 사용하여 RESTful API를 설계 및 구현하였다. 사용자 역할에 따른 접근제어, 세션 관리, 비밀번호 암호화 등 백엔드 프로그래밍 기법을 실습하였다.

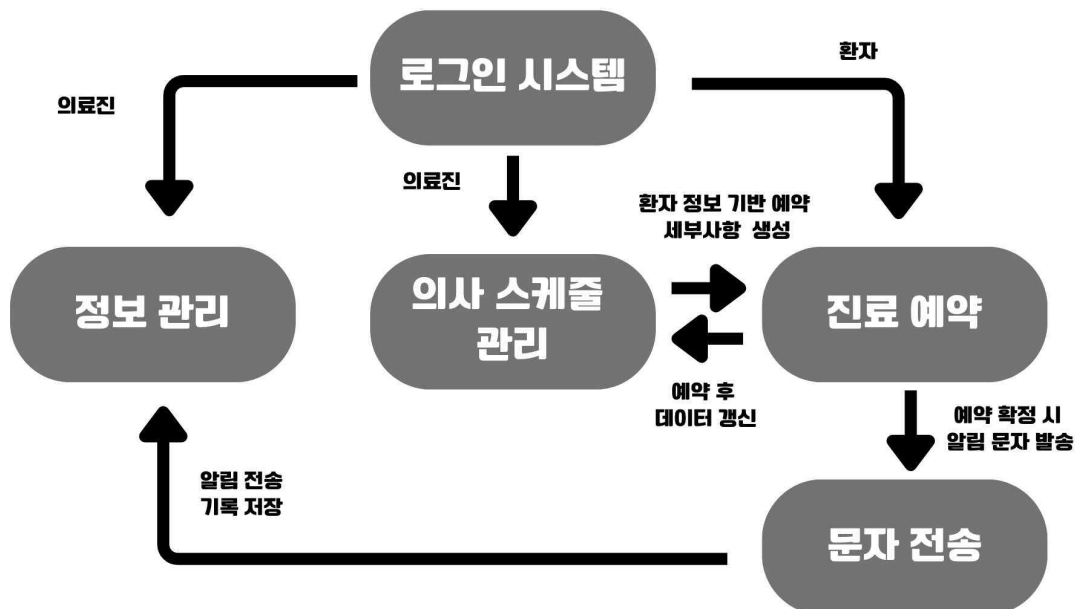
3) 웹 보안 기본 이론

개인정보 보호를 위해 비밀번호 해시화(BCrypt), 세션 타임아웃, 역할 기반 접근 제어, CSRF 및 XSS 방지 기법을 참고하였다.

4) 외부 API 연동 및 메시지 시스템

SMS 문자 알림 기능을 구현하기 위해 외부 문자 API Twilio를 조사하고 연동 방법을 익혔다.

5.2 소프트웨어 기능블록도 (개념설계)

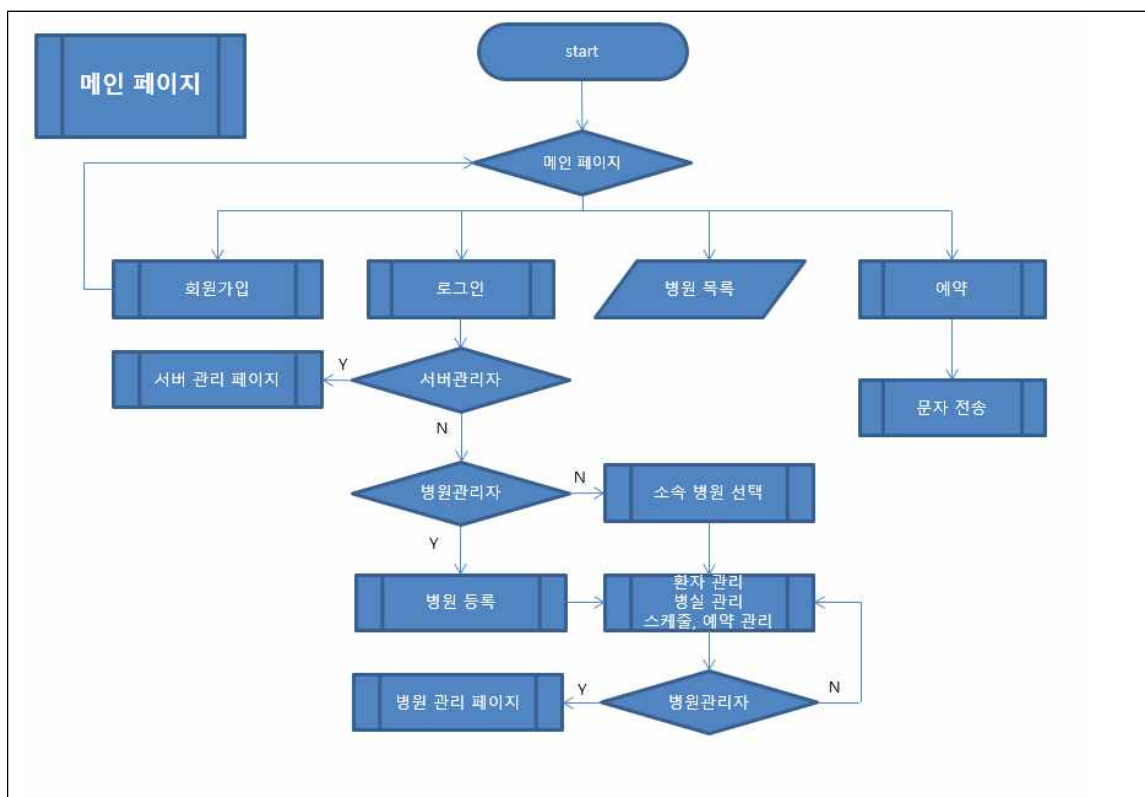


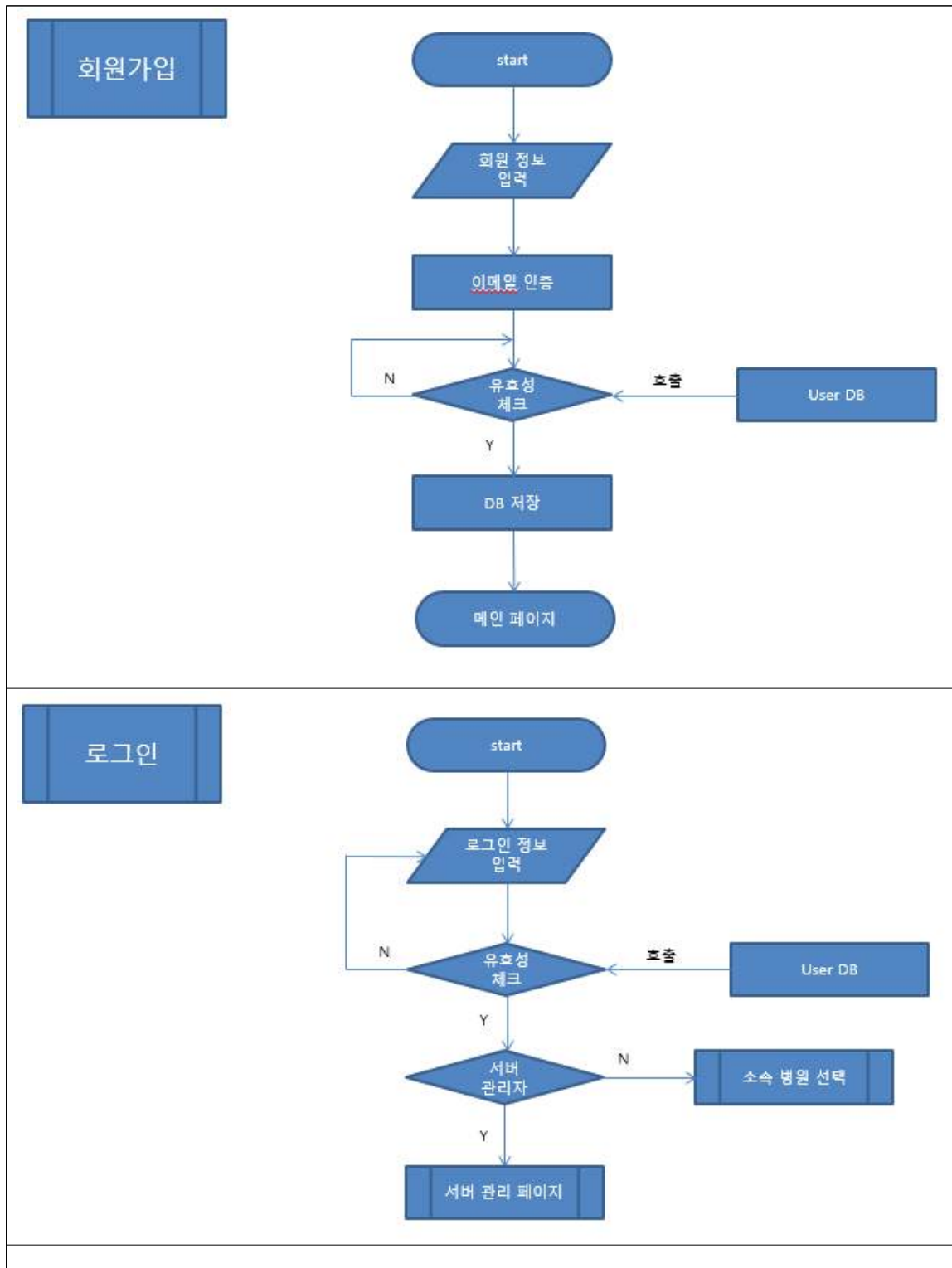
5.3 기능적 요구사항 명세서

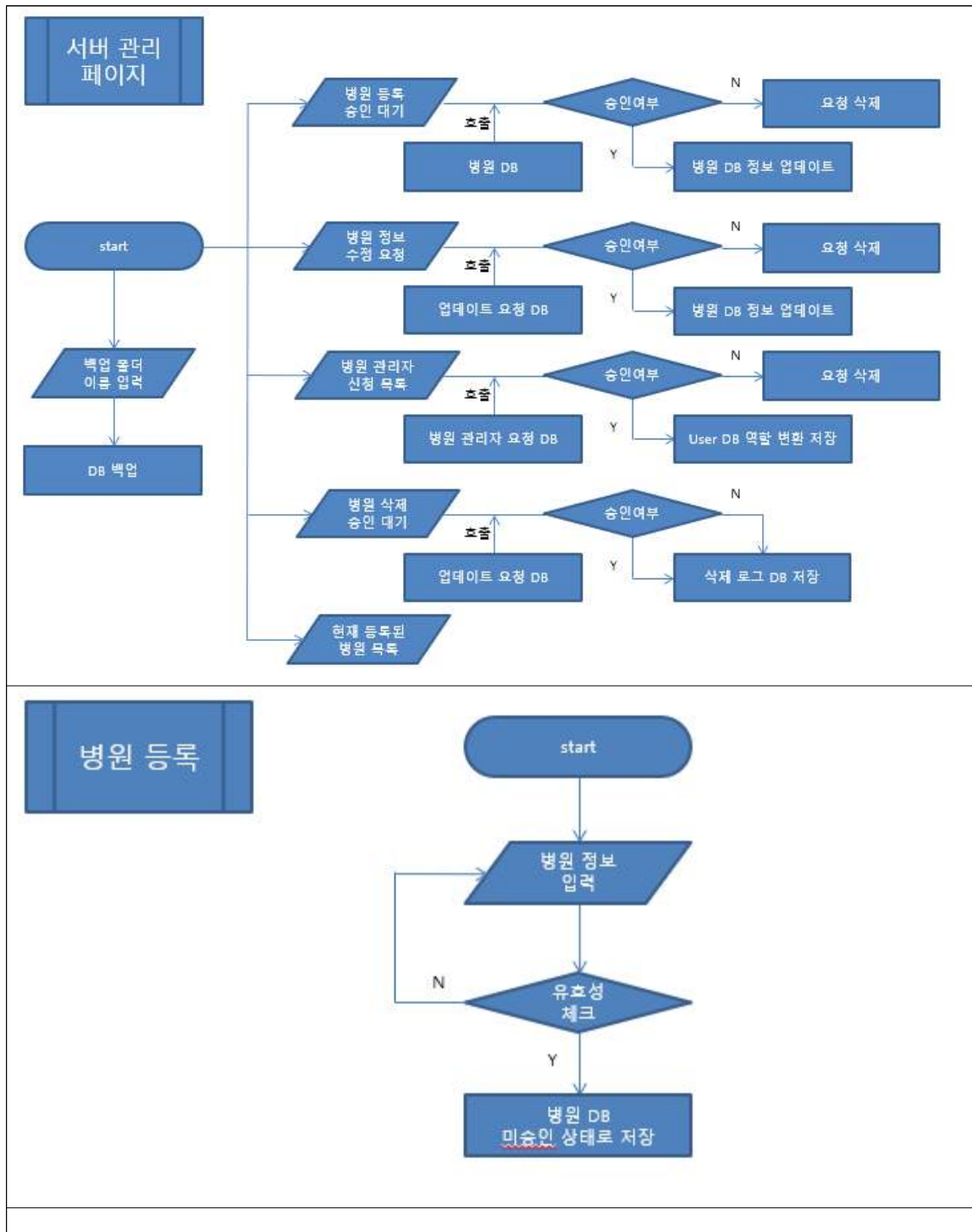
ID	요구사항	내용	설명	우선 순위
R_01	로그인 시스템	회원가입, 로그인, 역할 부여	회원가입 후 서버 관리자의 승인을 통해 의료진과 병원 관리자의 역할이 부여되며, 로그인된 사용자에게 해당 권한 기반으로 기능 제공한다.	1
R_02	사용자 권한 및 소속 병원 선택	역할 별 접근 제어	병원 관리자와 의료진, 환자 사용자의 로그인 후 인터페이스를 분리하여 자신이 소속한 병원의 각 역할에 맞는 기능과 화면이 제공되도록 하는 알고리즘 구현한다.	2
R_03	환자 개인정보 관리	환자 기본 개인정보 관리	환자의 인적사항(이름, 생년월일, 연락처 등)을 저장 및 수정이 가능하게 한다.	3
R_04	개인 정보 보호 기능	환자 정보 암호화, 접근제한	민감 정보는 암호화하여 저장하고, 일정 시간 비활성 시 자동 로그아웃, 접근 제한 등 보안 기능을 포함한다.	6
R_05	환자 진료정보 관리	환자의 진단 내용, 차트, 복약 기록 관리	의료진은 환자의 진료 정보(진단 내용, 진료 차트, 약물 복용 기록 등)을 저장하고 수정이 가능하게 한다.	4
R_06	환자 정보 조회	로그인 후 환자 정보 기록 확인 가능	로그인 후 진료기록을 열람할 수 있도록 한다. 환자는 본인의 정보만 조회 가능하며 수정이 불가하도록 한다.	5
R_07	병원 정보 등록	다양한 병원에서 사용 가능하도록 관리	병원 정보(병원 연락처, 주소 진료 시간, 위치 등)와 병실 구조, 병상 수 등을 서버에 등록하고 관리할 수 있도록 한다.	8
R_08	입퇴원 처리	입원 등록, 퇴원 처리, 병실 가용 여부	환자 입퇴원 시 병실 배정 및 해제를 처리하고 병실 가용 현황을 실시간으로 반영한다.	7
R_09	진료 예약	환자 진료 예약 신청 및 취소	환자가 로그인 후 직접 진료 예약을 신청하고 취소할 수 있도록 한다.	9
R_10	스케줄 등록	의사 진료 스케줄 입력	의사가 스케줄 등록 및 환자의 진료 예약 신청 수락이 가능하게 하고, 이를 실시간으로 의사의 진료 스케줄에 반영될 수 있도록 한다.	10

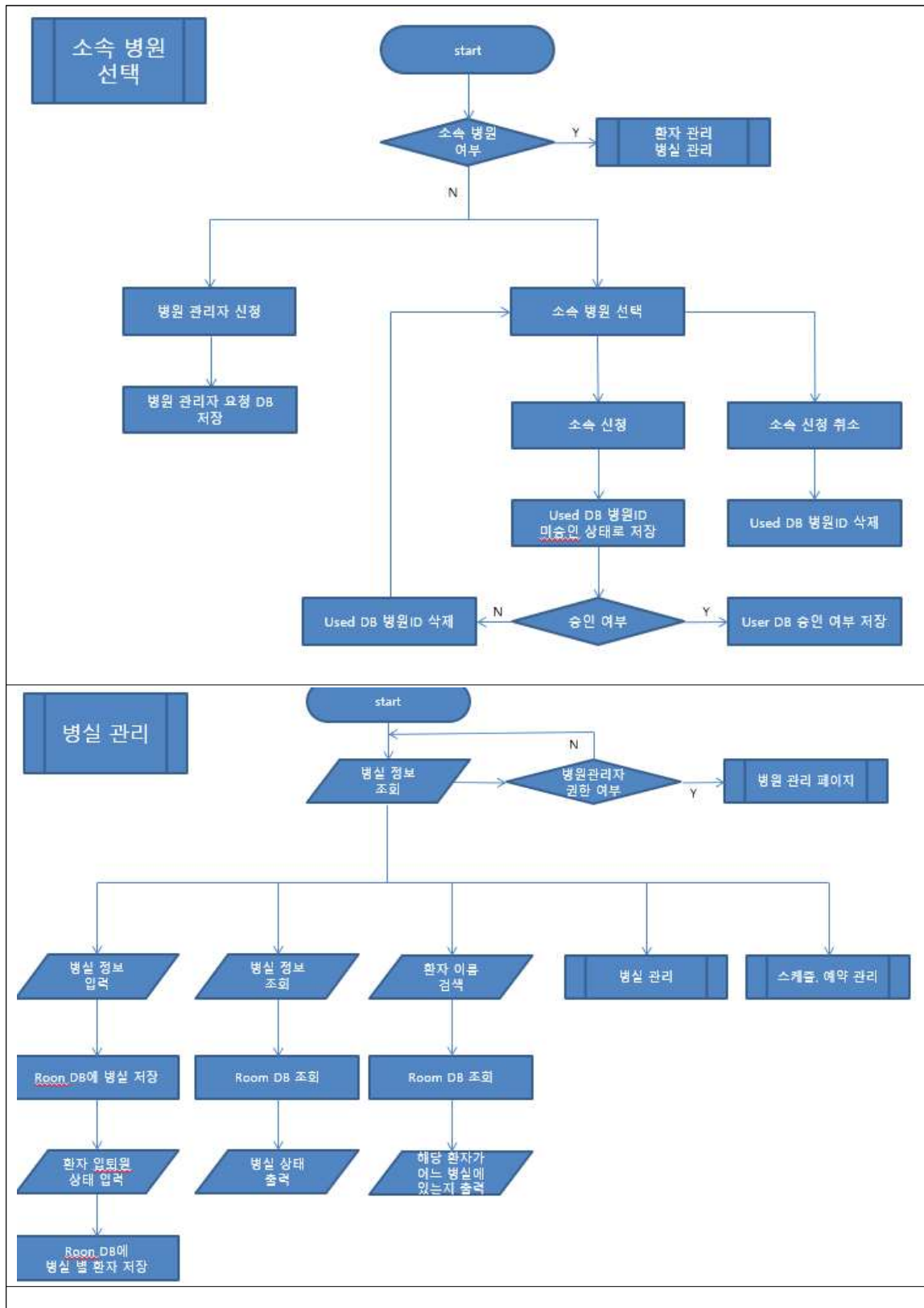
R_12	문자 전송 API 연동	환자에게 알림 문자 발송	회원가입 시 환자 핸드폰 번호로 진료 예약 확정 여부, 검사 전 준비사항 등의 내용을 SMS API를 통해 전송할 수 있어야 한다. 이를 통해 의료진의 반복 업무를 줄이고, 환자의 안내 누락이나 일정 착오를 예방할 수 있다.	11
R_13	데이터 백업 및 복구 시스템	DB 정기 백업 및 장애 복구 기능	주기적 자동 백업 스케줄링 및 복구 시스템을 마련하여 서버 안정성을 확보한다.	13

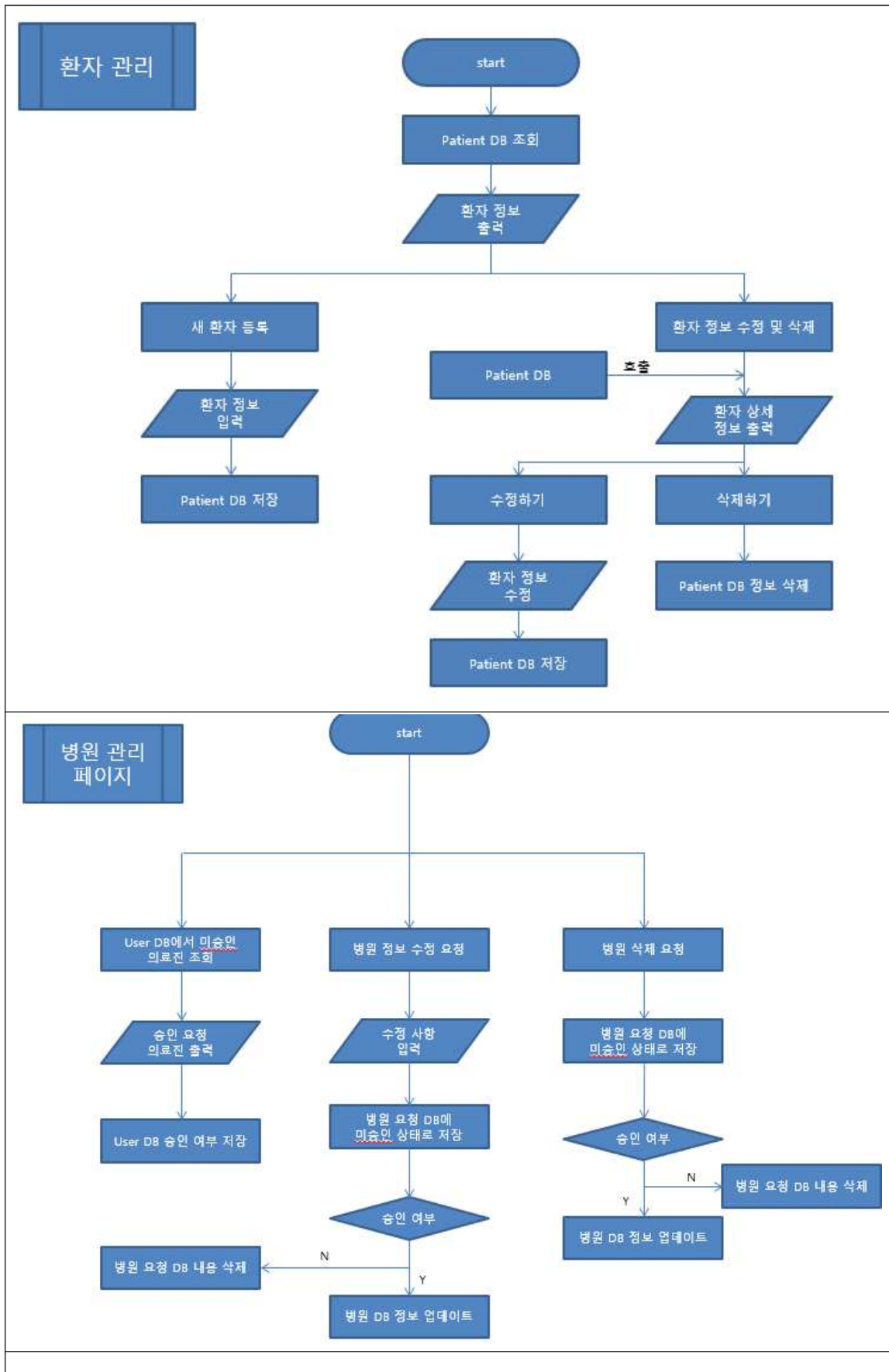
5.4 전체/부분 순서도 (상세설계)

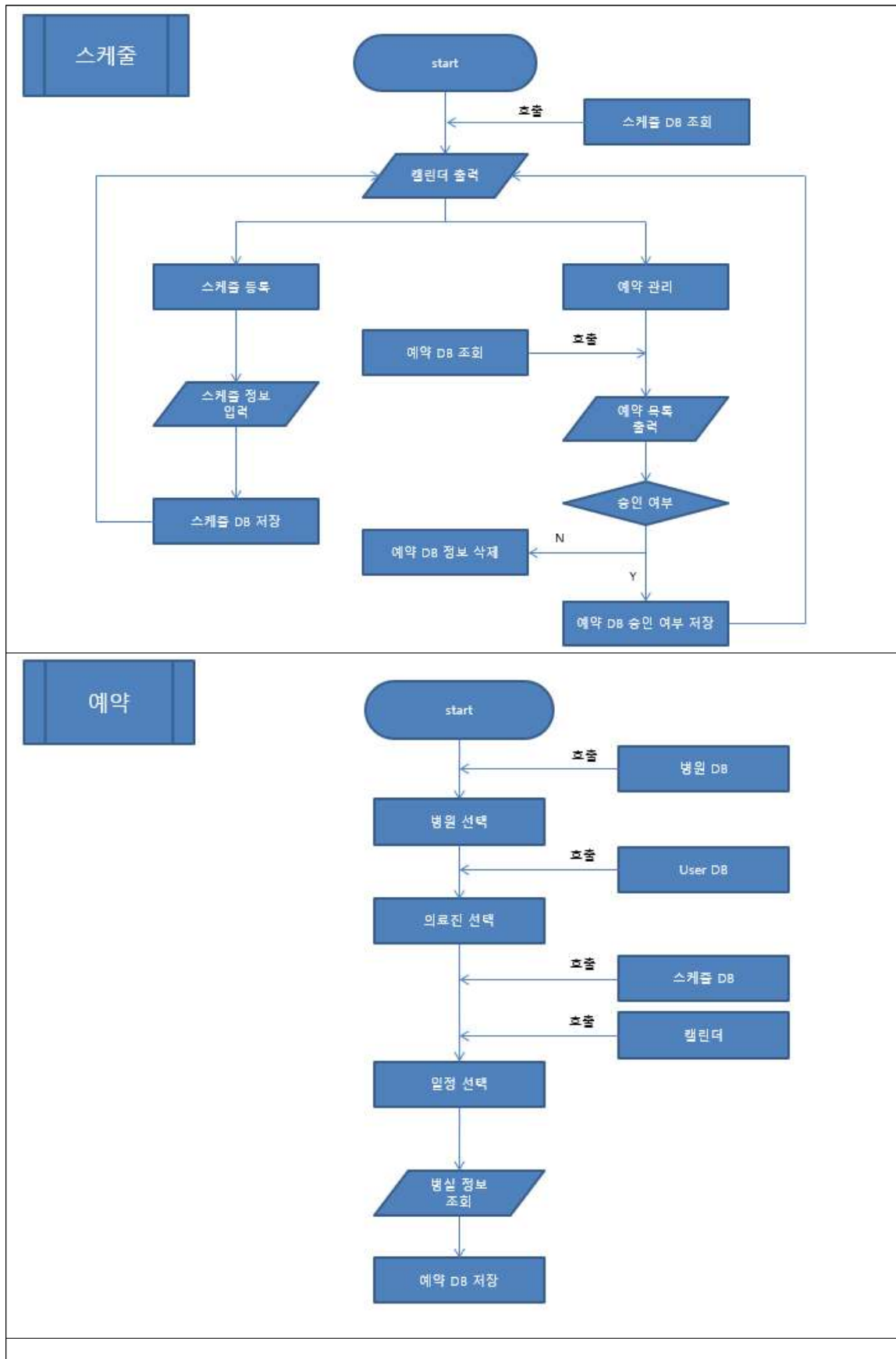


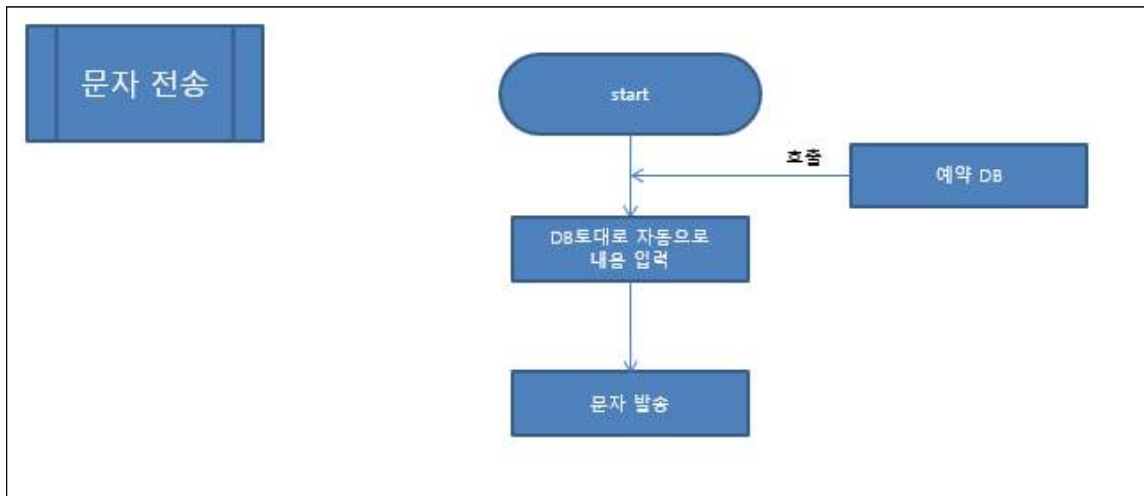












6. 제작(Implementation)

6.1 제작과정 (주요 기능)

1) 인증 및 접근제어 부분

<pre> const nodemailer = require('nodemailer'); require('dotenv').config(); const transporter = nodemailer.createTransport({ service: 'gmail', auth: { user: process.env.EMAIL_USER, pass: process.env.EMAIL_PASS } }); exports.sendEmail = (email, code) => { const mailOptions = { from: '"병원 관리자" <\${process.env.EMAIL_USER}>', to: email, subject: '이메일 인증번호', html: `<p>인증번호: <strong>\${code}</strong></p>` }; return transporter.sendMail(mailOptions); } </pre>	<pre> exports.ensureServerAdmin = (req, res, next) => { if (req.session.user && req.session.user.role === 'admin') { next(); } else { // 이전 페이지로 돌아가기 + 경고창 띄우기 res.send(` <script> alert('관리자 권한이 없습니다.'); </pre>
<pre> exports.isLoggedIn = (req, res, next) => { if (req.session.user) { next(); } else { // 세션이 만료되었거나 로그인 안 된 경우 res.render('redirectWithAlert', { message: '로그인이 만료되었습니다. 로그인 페이지로 돌아갑니다.', redirectUrl: '/auth/login' }); } } </pre>	<pre> const User = require('../models/user'); const hashHospital = async (req, res, next) => { try { const userId = req.session.user?._id; if (userId) { res.render('redirectWithAlert', { message: '로그인이 만료되었습니다. 로그인 페이지로 돌아갑니다.', redirectUrl: '/auth/login' }); } const user = await User.findById(userId); if (user user.hospitalId) { res.render('redirectWithAlert', { message: '소속 병원이 없습니다. 승인 후 이용해주세요.', redirectUrl: '/doctor' }); } } catch (err) { res.status(500).json({ error: '병원 확인 실패', details: err.message }); } } </pre>

- 이메일 인증 과정을 도입하여 보안을 강화하였다.
- 사용자 역할에 따른 접근 권한을 구현하기 위해 미들웨어를 사용하였다.
- 세션 관리를 통해 로그인 상태를 유지하며, 권한 없는 접근을 차단하였다.

2) 환자 등록 부분

<pre>// 새 환자 등록 exports.createPatient = async (req, res, next) => { try { const { name, birthdate, hospitalId, medicalRecords } = req.body; const newPatient = new Patient({ name, birthdate, hospitalId, medicalRecords, }); await newPatient.save(); res.status(201).json(newPatient); } catch (err) { next(err); } };</pre>	<pre><body> <h2>새 환자 정보 입력</h2> <form id="patientForm"> <label>이름: <input type="text" name="name" required> </label> <label>생년월일: <input type="date" name="birthdate" required> </label> <label>소속 병원 ID: <input type="text" name="hospitalId" id="hospitalId" readonly required> </label> <h3>의료 기록</h3> <label>기록 날짜: <input type="date" name="recordDate" required> </label> <label>문진 내용: <textarea name="symptoms" required></textarea> </label> <label>진단 내용: <textarea name="diagnosis" required></textarea> </label> <label>담당 의사 ID: <input type="text" name="doctorId" id="doctorId" readonly required> </label> <button type="submit">환자 정보 저장</button> </form></pre>
<pre><script> // 세션에서 병원 ID와 의사 ID 가져오기 async function loadSessionInfo() { const res = await fetch('/api/patient/session', { credentials: 'include' }); if (res.ok) { const data = await res.json(); document.getElementById('hospitalId').value = data.hospitalId; document.getElementById('doctorId').value = data.doctorId; } else { alert('세션 정보를 불러오지 못했습니다.');</pre>	

- 의료진 및 병원 관리자가 등록할 수 있도록 폼과 API를 구성하였다.
- 환자 등록 시 로그인 된 의료진의 병원ID와 의료진ID를 DB에서 가져와 폼에 자동으로 작성되게 하였다.

3) 병실 관리 부분

<pre>rooms = data.map((r) => { const roomNum = parseInt(r.roomNumber); const floor = r.floor; const x = ((roomNum % 100) - 1) * (roomWidth + margin) + margin; const y = (floor - 1) * (roomHeight + margin) + margin; return { x, y, number: r.roomNumber, patients: r.patients, capacity: r.capacity, hospitalId: r.hospitalId }; }); drawRooms(); });</pre>	<pre>function drawRooms() { ctx.clearRect(0, 0, canvas.width, canvas.height); rooms.forEach(room => { if (room.patients.length === 0) { ctx.fillStyle = '#66b66a'; } else if (room.patients.length < room.capacity) { ctx.fillStyle = '#fdd835'; } else { ctx.fillStyle = '#e53935'; } ctx.fillRect(room.x, room.y, roomWidth, roomHeight); ctx.strokeStyle = 'black'; ctx.strokeRect(room.x, room.y, roomWidth, roomHeight); ctx.fillStyle = 'white'; ctx.font = '14px Arial'; ctx.fillText(`\${room.number}`, room.x + 10, room.y + 20); ctx.fillText(`\${room.patients.length}/\${room.capacity}명`, room.x + 10, room.y + 40); }); canvas.addEventListener('click', e => { const rect = canvas.getBoundingClientRect(); const x = e.clientX - rect.left; const y = e.clientY - rect.top; for (const room of rooms) { if (x > room.x && x < room.x + roomWidth && y > room.y && y < room.y + roomHeight) { openDialog(room); break; } } }); }</pre>
--	--

```
const mongoose = require('mongoose');

const roomSchema = new mongoose.Schema({
  hospitalId: {
    type: String,
    required: true,
  },
  floor: {
    type: Number,
    required: true,
  },
  roomNumber: {
    type: String,
    required: true,
  },
  capacity: {
    type: Number,
    required: true,
  },
  patients: {
    type: [String], // 환자 이름 목록
    default: [],
  },
  lastUpdated: {
    type: Date,
    default: Date.now,
  },
});

module.exports = mongoose.model('Room', roomSchema);
```

```
// 입원
assignPatient: async (req, res) => {
  const { hospitalId, roomNumber, patientName } = req.body;

  const room = await Room.findOne({ hospitalId, roomNumber });
  if (!room) return res.status(404).json({ error: '병실 없음' });
  if (room.patients.length >= room.capacity) return res.status(400).json({ error: '정원 초과' });

  room.patients.push(patientName);
  room.lastUpdated = new Date();
  await room.save();

  res.json({ status: '입원 완료' });
},

// 퇴원
dischargePatient: async (req, res) => {
  const { hospitalId, roomNumber, patientName } = req.body;

  const room = await Room.findOne({ hospitalId, roomNumber });
  if (!room) return res.status(404).json({ error: '병실 없음' });

  const index = room.patients.indexOf(patientName);
  if (index === -1) return res.status(400).json({ error: '환자 없음' });

  room.patients.splice(index, 1);
  room.lastUpdated = new Date();
  await room.save();

  res.json({ status: '퇴원 완료' });
},
```

- 병실 배치도를 시각화하여 직관적인 병실 현황을 제공하였다.
- 병원 ID를 기준으로 병실 데이터를 분리해 관리하며, 입퇴원 상태가 실시간으로 반영된다.
- 병실 번호를 층수와 호수로 분리하여 좌표 계산 후 시각화 하여 정렬하였다.

4) 예약 기능 부분

```
// 특정 의사의 스케줄 조회
exports.getSchedulesByDoctor = async (req, res) => {
  try {
    const doctorObjectId = new mongoose.Types.ObjectId(req.params.doctorId);
    const schedules = await Schedule.find({
      doctorId: doctorObjectId,
      date: { $gte: new Date() }
    })
    .sort({ date: 1 })
    .populate('doctorId', 'name');

    const events = schedules.map(schedule => ({
      id: schedule._id,
      title: schedule.isHoliday ? '휴진' : '진료 가능',
      start: schedule.date,
      end: schedule.date,
      extendedProps: {
        status: schedule.status || (schedule.isHoliday ? 'unavailable' : 'available'),
        department: schedule.department,
        availableTime: schedule.availableTime
      }
    }));
    res.json(events);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
};
```

```
// 예약 생성
exports.createReservation = async (req, res) => {
  const { patientId, patientName, phoneNumber, hospitalId, doctorId, scheduleId, reservedTime } = req.body;
  console.log('서버에서 받은 hospitalId:', hospitalId);
  console.log('서버에서 받은 scheduleId:', scheduleId);

  // 1. 병원, 스케줄 유효성 검사
  const hospital = await Hospital.findById(hospitalId);
  console.log('서버에서 찾은 hospital:', hospital);
  if (!hospital || !hospital.approved) {
    return res.status(400).json({ error: '등록되지 않은 병원입니다.' });
  }
  const schedule = await Schedule.findById(scheduleId);
  console.log('서버에서 찾은 schedule:', schedule);
  if (!schedule) {
    return res.status(400).json({ error: '해당 의료진의 스케줄이 없습니다. 예약 불가.' });
  }

  // (추가) 예약 가능 시간인지 체크해도 됨

  const newReservation = new Reservation({ patientId, patientName, phoneNumber, hospitalId, doctorId, scheduleId, reservedTime });
  const saved = await newReservation.save();

  // 예약이 들어온 스케줄과 상태를 'pending'으로 변경
  schedule.status = 'pending';
  await schedule.save();

  res.status(201).json(saved);
}
```

```
Fullcalendar Standard Bundle v6.1.10
Docs & License: https://fullcalendar.io/docs/initialize-globals
(c) 2023 Adam Shaw

/*
var FullCalendar=function(e){var t,n,r,i,s,o,a,l,c,d={
function f(e,t){for(var n in t)e[n]=t[n];return e}function g(e){var t
"key"==o?i=n[o]:"ref"==o?s=n[o]:a[o]=n[o];if(arguments.length>2&&(a,
null!=e.defaultProps)for(o in e.defaultProps)void 0===a[o]&&(a[o]=e.d
props[t,key:i,ref:s,_:k,null,_:null,_:b:0,_:e:null,_:d:void 0,_:c:n
vnode&&n.vnode(a,a)}function v(){return{current:null}}function y(e){
[t]=null==n?"":n;"number"!=typeof n||h.test(t)?n:n+"px"}function E(e,t
("string"!=typeof r&&(e.style.cssText=r),r)for(t in r)n&&t in n||
("o"===t[0]&&"n"===t[1])s=t[1]s=t[1].replace(/Capture$/,""),t=t.toLowerCase
e.addEventListener(t,s?A:S,s);e.removeEventListener(t,s?A:S,s);else i
sName$/,"s");else if("width"!=t&&"height"!=t&&"href"!=t&&"list"!=
break e)catch(e){}function"!=typeof n||(null==n||1!==n&&1==t.index
return this;if(!t.nodeType||n.event?n.event(e):e)}finally{v(s=1)}function
```



- 환자가 실시간으로 의료진 스케줄을 확인하고 예약 가능한 시간대를 선택할 수 있도록 구현하였다.
- 예약 상태에 따라 UI와 예약 가능 여부를 동적으로 반영하도록 설계하였다.
- 의료진의 스케줄을 캘린더를 사용하여 직관적으로 보이도록 설계하였다.

기능 모듈 간 인터페이스 구현

각 모듈은 RESTful API 형태로 통신한다. 예를 들어, 진료 예약 모듈은 예약 승인 시 문자 알림 모듈에 API 요청을 보내 알림 메시지를 자동 발송하도록 연결하였다.

또한, 사용자 권한 검증 모듈은 모든 민감 기능 접근 시 미들웨어로 호출되어 역할 기반 접근 제어를 수행한다.

참고한 라이브러리 및 가공 적용 과정

Express.js를 활용해 서버 및 라우팅을 구성하였다.

Mongoose 라이브러리를 통해 MongoDB와의 데이터 연동을 처리하였다.

bcrypt 라이브러리로 비밀번호 암호화를 구현하였으며, 이를 커스텀 함수로 감싸 보안 강화를 도모하였다.

외부 SMS API는 테스트용 무료 API를 연동하여 문자 발송 기능을 구현하였다.

6.2 제작시 문제점 및 개선사항

1) 비동기 처리 오류

- 개발 초기 데이터베이스 조회와 API 호출이 비동기로 동작하면서 순서가 맞지 않아 오류가 발생하였다. 이는 async/await 문법을 학습하고 적용하여 각 비동기 함수에서 예외 처리를 추가하여 흐름 제어를 명확히 하면서 해결하였다.

2) 환자의 정보 확인 기능

- 초기에는 환자가 직접 회원가입하고 자신의 정보를 직접 확인할 수 있도록 설계하려 하였으나, 정보의 정확성 및 개인정보 보호 측면에서 우려가 있어, 의료진이 직접 환자를 등록하는 방식으로 변경하면서 환자의 로그인 기능과 정보 열람 기능이 삭제되었다.

3) 증상 기반 진료과 추천 AI 기능 삭제

- AI 기반으로 환자의 증상을 분석해 진료과를 추천하는 기능을 설계하려 하였으나,

의료법 관련 규정상 자가진단 및 진단 보조 도구가 의료인의 판단을 대신해서는 안된다는 규정이 있어 문제가 되었다. 법적 논란을 피하고 서비스 안정성을 확보하기 위해 해당 기능을 삭제하였다.

4) 무료 SMS API의 한계

- 문자 알림 기능 구현을 위해 무료 SMS API를 사용했지만, 발송 제한, 발신 번호 고정, 인증 지연 등의 문제가 발생했다.

→ 테스트 수준으로는 충분했지만, 실제 상용화를 위해서는 유료 서비스 도입이 필요하다는 점을 인식하게 되었다

6.3 지도교수 지도내용 및 지적사항 조치결과

7. 시험

1. 회원가입, 로그인 부분

1) 입력하지 않았을 시 : 기대 결과와 같다.

The first screenshot shows the registration form with validation messages: '이름:이름을 입력하세요' (Name: Please enter your name), '이메일:이 이메일을 작성하세요' (Email: Please enter this email), and '비밀번호:비밀번호(6자 이상, 영문+숫자)' (Password: Password (6 characters or more, alphanumeric)). The second screenshot shows the form with the name and email filled, but the password field is empty, with a message '비밀번호:비밀번호(6자 이상, 영문+숫자)'. The third screenshot shows the form with the name and email filled, but the password field is empty, with a message '비밀번호:비밀번호(6자 이상, 영문+숫자)'.

2) 다른 이메일로 인증 요청 시 : 기대 결과와 같다.

The dialog box has a title 'localhost:3000 내용:' (localhost:3000 Content:). The main text says '병원 이메일만 인증 가능합니다.' (Only hospital email can be verified.). There is a '확인' (Confirm) button. Below the dialog box, the registration form is visible with the name '이현영' (Lee Hyun-yeong) and email 'ggamci2001@naver.com' entered, and a '인증번호 발송' (Send verification code) button.

3) 이미 등록된 이메일 입력 시 : 기대 결과와 다르다.

```
const existingUser = await User.findOne({ email });
if (existingUser) {
  return res.render('signup', { error: '이미 등록된 이메일입니다.' });
}
```

localhost:3000 내용:

인증번호가 전송되었습니다!

확인

이름: 이현영

이메일: ggamci2001@gmail.com

인증번호 발송

- 로직은 있으나 조건문이 잘 작동하지 않는 것으로 보이며, 잘 해결되지 않았다.

4) 인증하지 않을 시 : 기대 결과와 같다.

회원가입

이메일 인증을 완료해주세요.

이름: 이름을 입력하세요

5) 잘못된 비밀번호 입력 시 : 기대 결과와 같다.

회원가입

비밀번호는 영문+숫자 포함 6자 이상이어야 합니다.

- 그 외 정상 입력은 기대 결과대로 작동하며, 로그인 부분도 동일하다.

2. 권한 별 접근

1) 권한이 없는 의료진이 다른 사이트 가려고 할 때 : 기대 결과와 같다.

localhost:3000/admin

localhost:3000 내용:
관리자 권한이 없습니다.

확인

localhost:3000/afterlogin/rooms

localhost:3000 내용:
소속 병원이 없습니다. 승인 후 이용해주세요.

확인

localhost:3000/manager

localhost:3000 내용:
병원 관리자 권한이 없습니다.

2) 세션이 끝나 자동 로그아웃 : 기대 결과와 같다.

localhost:3000/doctor ☆

localhost:3000 내용:

로그인이 완료되었습니다. 로그인 페이지로 돌아갑니다.

확인

- 하지만 새로고침 전까지는 동작하지 않는다는 문제가 있다.

3. 환자 정보 부분

- 1) 새 환자 정보 입력 시 빈칸 : 기대 결과와 같다.

새 환자 정보 입력

이름: 생년월일: 연도-월-일	생년월일: 연도-월-일 소속 병원 ID:
---	---

이 입력란을 작성하세요

이 입력란을 작성하세요.

- 환자 정보 수정 시에도 빈칸 시 동일하게 나온다.
- 그 외 정상 입력은 기대 결과로 작동한다.

4. 병실 관리 부분

- 1) 모든 정보를 입력하지 않을 시 : 기대 결과와 같다.

localhost:3000 내용:

모든 정보를 정확히 입력해주세요.

확인

병원 ID: e847488d0f55627abae789f5

총 수: 3 총당 방 수: 4 병당 4

병실 정보 입력 병실 정보 조회

- 2) 입원한 환자가 없을 때 검색할 시 : 기대 결과와 같다.

이현영 검색

❌ 이현영님은 입원 중이 아닙니다.

- 3) 환자 이름 입력 없이 입퇴원 : 기대 결과와 같다.

localhost:3000 내용:

이름을 입력하세요.

확인

이현영 검색

❌ 이현영님은 입원 중이 아닙니다.

병실 101

수용 인원:0/4명

현재 환자:

환자 이름

입원 퇴원 당기

localhost:3000 내용:

퇴원시킬 환자 이름을 입력하세요.

확인

이현영 검색

❌ 이현영님은 입원 중이 아닙니다.

병실 101

수용 인원:0/4명

현재 환자:

환자 이름

입원 퇴원 당기

- 그 외 정상 입력은 기대 결과로 작동한다.

5. 예약 및 스케줄 부분

1) 빈칸 시 : 기대 결과와 같다.

스케줄 등록

병원
서울 병원

의료진
강현욱

진료과
진료과 선택

날짜  목록에서 항목을 선택하세요.

2) 예약 승인 후 취소 시 : 기대 결과와 다르다.

예약 관리

예약일	시간	환자	상태	작업
2025. 6. 12.	09:37 ~ 10:37	-	취소됨	
2025. 6. 12.	11:37 ~ 13:37	-	예약 가능	삭제

- 원래는 취소 후 예약 가능으로 바뀌거나 삭제돼야 하는데 그렇게 되지 않았다.

3) 환자 예약 신청 시 빈칸 : 기대 결과와 같다.

예약 날짜
2025. 6. 13.

예약 시간
10:43 ~ 00:43

환자 이름

전화번호  이 입력란을 작성하세요.

010 숫자 8자리

하이픈(-) 없이 숫자 8자리를 입력하세요.

8. 평가

8.1 정량적/정성적 목표달성도 평가

항목	목표값 (4장 항목별 목표값 참조)	달성률 (%)	비고
프로그램의 구현	병원 웹 서버 시스템을 구현	90	주요 기능 대부분 구현 완료
인터페이스	사용자 친화적인 인터페이스 구현	70	직관적인 인터페이스는 구현 완료 다만 디자인은 최소화함.
모듈화	기능별 모듈화, 미들웨어, API 방식 설계로 유지보수가 용이한 코드 구현	95	체계적인 폴더 구조로 나눔
구현 기능	기능적 요구사항 명세서의 내용	80	

8.2 현실적 제한요소 달성도 평가

현실적 제한 요소들	목표	달성결과
경제	- 자체 서버 구축 및 운영에 드는 초기 비용이 크며, 보안 강화와 외부 API 사용 시 지속적인 비용 부담이 발생한다. - 특히 병원 규모가 작을수록 시스템 도입 비용 대비 효율성이 낮아, 비용 부담이 더욱 클 수 있다.	로컬 서버로 테스트 환경을 구성하였고, 무료 버전 외부 API를 도입하였으나, 테스트 수준에 한정된다.
편리	- 고령의 의료진이나 디지털 기기에 익숙하지 않은 사용자는 시스템 학습에 어려움을 겪을 수 있다. - 의료진과 환자 각각의 다양한 사용 환경을 모두 만족시키기 위한 대응이 제한적일 수 있다. - 응급 상황에서 웹 기반 시스템의 작동 지연 또는 장애 발생 시 신속한 대응이 어려울 가능성이 있다.	역할별 화면 분리와 단순한 UI로 디자인 측면에선 부족하나 기본적인 편의성은 확보하였다.
윤리	- 개인정보와 진료 기록 등 매우 민감한 데이터는 철저한 암호화와 접근 통제를 통해 보호되어야 한다.	bcrypt 암호화, 미들웨어 기반 접근 제한 구현을 완료하였다.
사회	- 고령자, 장애인, 외국인 환자 등 디지털 접근성이 낮은 사회적 약자 계층이 서비스에서 소외될 우려가 있다.	기본 웹 구조를 최대한 단순하고 직관적으로 만들었다.

8.3 기능적 요구사항 달성도 평가

ID	요구사항	내용	설명	우선 순위	달성률 (%)	비고
R_01	로그인 시스템	회원가입, 로그인, 역할 부여	회원가입 후 서버 관리자의 승인을 통해 의료진과 병원 관리자의 역할이 부여되며, 로그인 된 사용자에게 해당 권한 기반으로 기능 제공한다.	1	95	
R_02	사용자 권한 및 소속 병원 선택	역할 별 접근 제어	병원 관리자와 의료진, 환자 사용자의 로그인 후 인터페이스를 분리하여 자신이 소속한 병원의 각 역할에 맞는 기능과 화면이 제공되도록 하는 알고리즘 구현한다.	2	95	
R_03	환자 개인정보 관리	환자 기본 개인정보 관리	환자의 인적사항(이름, 생년월일, 연락처 등)을 저장 및 수정이 가능하게 한다.	3	100	
R_04	개인 정보 보호 기능	환자 정보 암호화, 접근제한	민감 정보는 암호화하여 저장하고, 일정 시간 비활성 시 자동 로그아웃, 접근 제한 등 보안 기능을 포함한다.	6	90	
R_05	환자 진료정보 관리	환자의 진단 내용, 차트, 복용 기록 관리	의료진은 환자의 진료 정보(진단 내용, 진료 차트, 약물 복용 기록 등)을 저장하고 수정이 가능하게 한다.	4	95	
R_06	환자 조회	로그인 후 환자 정보 기록 확인 가능	로그인 후 진료기록을 열람할 수 있도록 한다. 환자는 본인의 정보만 조회 가능하며 수정이 불가하도록 한다.	5	80	
R_07	병원 등록	다양한 병원에서도 사용 가능하도록 관리	병원 정보(병원 연락처, 주소 진료 시간, 위치 등)와 병실 구조, 병상 수 등을 서버에 등록하고 관리할 수 있도록 한다.	8	100	
R_08	입퇴원 처리	입원 등록, 퇴원 처리, 병실 가용 여부	환자 입퇴원 시 병실 배정 및 해제를 처리하고 병실 가용 현황을 실시간으로 반영한다.	7	100	
R_09	진료 예약	환자 진료 예약 신청 및 취소	환자가 로그인 후 직접 진료 예약을 신청하고 취소할 수 있도록 한다.	9	80	
R_10	스케줄 등록	의사 진료 스케줄 입력	의사가 스케줄 등록 및 환자의 진료 예약 신청 수락이 가능하게 하고, 이를 실시간으로 의사의 진료 스케줄에 반영될 수 있도록 한다.	10	80	

R_12	문자 전송 API 연동	환자에게 알림 문자 발송	회원가입 시 환자 핸드폰 번호로 진료 예약 확정 여부, 검사 전 준비사항 등의 내용을 SMS API를 통해 전송할 수 있어야 한다. 이를 통해 의료진의 반복 업무를 줄이고, 환자의 안내 누락이나 일정 착오를 예방할 수 있다.	11	70	
R_13	데이터 백업 및 복구 시스템	DB 정기 백업 및 장애 복구 기능	주기적 자동 백업 스케줄링 및 복구 시스템을 마련하여 서버 안정성을 확보한다.	13	80	

8.2 문제점 및 해결방안

1. 접근 권한 관련

- 1) 세션이 만료된 상태에서도 이전 로그인 정보를 기반으로 특정 페이지 접근이 가능했던 문제가 있었고, 이를 방지하기 위해 모든 주요 페이지 접근 시 세션과 로그인 상태를 재검증하는 미들웨어를 추가하여 보안을 강화하였다.
- 2) 세션이 만료되어도 화면을 새로고침하지 않으면 그 화면에서 기능을 계속 사용할 수 있는 문제가 있었다. 세션 만료 감지 기능이 있음에도 이 문제는 해결하지 못하였다.

2. 병원 관리자 기능 관련

- 1) 병원 이름이 중복될 경우에도 병원이 등록되는 문제가 있었고, 이를 방지하기 위해 병원명 중복 여부를 검사하여 등록할 수 없도록 제한하였다.
- 2) 병원 정보 수정 요청 시 기존 정보와 수정 요청 정보가 관리자 화면에서 구분되지 않아 확인이 불편했으며, 이를 해결하기 위해 기존 정보와 수정 요청 정보를 함께 표시하도록 UI를 개선하였다.
- 3) 병원 관리자가 아직 병원을 등록하지 않았음에도 소속 병원 승인 대기 목록이나 권한 양도 기능이 노출되어 혼란을 초래했으며, 병원 등록이 완료되지 않은 경우에는 병원 관리자 화면에 등록 기능만 노출되도록 제한하여 흐름을 명확히 하였다.
- 4) 병원 관련 서버 관리자 기능(등록 승인, 관리자 신청 목록, 수정/삭제 승인 등)이 분산되어 사용이 불편했으며, 이를 하나의 통합 화면에서 처리할 수 있도록 구성하고 등록된 병원 목록도 함께 확인할 수 있도록 개선하였다.

3. 병실 관리 기능 관련

- 1) 병실 관리 모듈에서 병원 ID가 MongoDB의 ObjectId 형식으로 저장되어 사용자 입력값과 일치하지 않아 병실 생성 및 조회에서 오류가 발생했으며, 이를 병원 ID를 문자열로 통일하여 처리하였다.
- 2) 병실 번호 정렬이 실제 층수와 호수 기준과 맞지 않아 화면 레이아웃이 어긋났고, 병실 번호에서 층수와 호실 정보를 분리하여 좌표를 계산함으로써 정렬 문제

를 해결하였다.

- 3) 환자 입퇴원 처리 후 화면에 즉시 반영되지 않아 불편함이 있었으며, 입퇴원 처리 이후 병실 목록을 자동 새로고침하여 반영되도록 개선하였다.

4. 환자 정보 관리 기능 관련

- 1) 환자 차트 모듈에서 병원 간 접근 제어가 되지 않아 다른 병원의 의료진이 환자 정보를 조회할 수 있는 보안 문제가 있었고, 병원이 다를 경우 접근 요청과 승인을 거쳐야 열람할 수 있도록 제어 로직을 추가하였다.
- 2) 의료 기록을 환자와 별도의 테이블로 구성했을 때 환자별 진료 이력을 빠르게 조회하기 어려웠으며, 의료 기록을 환자 정보 안에 함께 저장하는 구조로 변경하여 관리의 효율성을 높였다.

5. 예약, 스케줄 기능 관련

- 1) 의료진 스케줄 등록 모듈에서 병원 선택 후에도 의료진 목록이 자동으로 표시되지 않아 매번 직접 확인해야 했으며, 병원 선택 이벤트를 감지하여 의료진 목록을 자동으로 불러오도록 개선하였다.
- 2) 스케줄 등록 시 휴일 여부나 시간 입력 없이도 등록이 가능해 비정상적인 데이터가 입력되는 문제가 있었고, 필수 항목 입력 여부를 철저히 검증하여 입력값 누락 시 등록이 불가능하도록 로직을 강화하였다.
- 3) 환자 예약 신청 모듈에서는 이미 예약된 시간대에도 중복 예약이 가능하여 충돌 및 혼란이 발생할 수 있었고, 예약 상태를 명확히 구분하여 예약 가능한 시간만 선택할 수 있도록 제한하는 기능을 추가하였다.
- 4) 병원 검색 기능에서 존재하지 않는 병원명을 입력했을 경우 아무런 안내 없이 빈 화면만 표시되어 사용자가 혼란을 겪었으며, 검색 결과가 없을 경우 "등록되지 않은 병원입니다"라는 메시지를 출력하여 안내하도록 개선하였다.

9. 추진체계

본 과제의 추진을 위한 팀은 4인으로 구성되며 다음과 같은 역할을 담당한다. 이론 연구 단계에서부터 과제 종료시점까지 협력과 보완을 통해 팀의 목표를 달성할 수 있도록 한다.

팀장 : 이현영(과제 총괄 및 보고서 작성)

팀원 : 강현욱(외부 API 연동 개발 및 테스트)

팀원 : 김민준(기능별 테스트 및 디버깅)

팀원 : 김승호(주요 기능 구현 총괄)

10. 설계 추진 일정: 2012년 8월 ~ 1012년 12월

수행 내용		일정 (1주 단위)					
		1	2	3	4	5	6
목표와 기준 설정	- 설계목표 설정						
합성	- 기능 블록 구성 - 기능별 구현방법 결정 - 적용할 이론 및 기술						
분석	- 세부 기능 블록도 - 목표달성가능성 확인						
제작	- 프로그램 코딩						
시험/평가	- 시험 및 검증 - Trouble shooting - 재설계						
결과	- 결과보고 및 시연						

11. 결론

본 병원 웹 시스템 프로젝트에서는 병실 관리, 환자 차트, 병원 등록 및 승인, 진료 예약 및 스케줄 관리 등 주요 기능들을 구현하며 사용자 중심, 운영 효율성, 시스템 안정성 확보에 중점을 두었다.

로그인 및 회원가입 로직에서는 사용자 인증과 역할 기반 접근 제어 구현에 집중하였다. 이메일 인증 절차를 도입하여 보안을 강화했으며, 회원가입 시 역할 구분과 병원 소속 확인을 통해 시스템 무결성을 확보하였다. 다만 세션 만료 시 화면 갱신 없이 기능이 계속 사용되는 문제는 완전한 해결이 어려웠으나, 이를 인지하고 개선 방향을 모색한 점도 의미 있는 경험이었다.

병실 관리 모듈에서는 캔버스를 활용한 시각적 배치와 실시간 입퇴원 기능을 구현했고, 환자 차트 모듈에서는 의료정보 보안과 접근 권한 제어를 강화하였다. MongoDB 기반의 데이터 저장과 조회, 권한 분리 등의 백엔드 핵심 기능을 직접 설계하며 데이터 구조와 UI 반영, 보안 처리, 실시간 데이터 반영에 대한 종합적인 이해를 높였다.

병원 등록 및 관리, 소속 병원 승인 파트에서는 약 95% 이상의 기능을 완성하고 역

할별 승인 기반 처리 흐름과 화면 구성을 명확히 하여 시스템의 일관성과 사용자 편의성을 강화하였다. 실제 사용자 경험과 업무 흐름을 반영해 로직을 개선하고, 병원 관리자와 서버 관리자의 화면을 분리하여 운영 효율성을 높였다.

의료진 스케줄 등록 및 환자 예약 모듈은 달력 기반 UI를 활용하여 스케줄 즉시 반영과 중복 없는 예약 신청을 가능하게 설계했다. 개발 과정에서 입력값 검증, 중복 방지, 병원과 의료진 연동 등 다양한 문제를 해결하며 데이터 정합성과 사용자 편의성 간 균형을 고려하는 역량을 키웠다.

이번 프로젝트를 통해 실무에서 요구되는 승인 절차, 역할 관리, 보안 설계, UI/UX 개선 등 병원 정보 시스템 개발 전반에 대한 실질적인 경험과 역량을 쌓을 수 있었다.

부록

<프로그램 소스코드>

- 라인별 커멘트 작성
- 사이버캠퍼스에 업로드
- 안되면 대용량첨부로 이메일로 전송